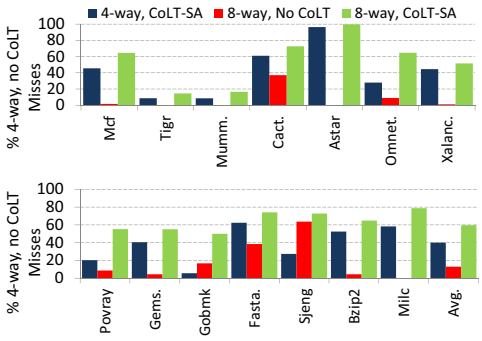


Figure 20: Percentage of baseline misses eliminated by CoLT-SA when increasing associativity.

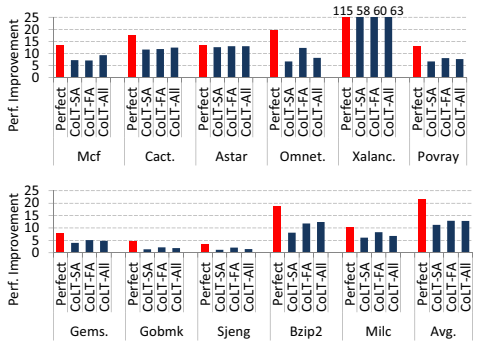


Figure 21: CoLT-SA, CoLT-FA, and CoLT-All performance improvements compared to perfect TLBs.

marks. This also implies that CoLT strategies have the potential to significantly improve performance.

Fortunately, Figure 21 shows that all of the CoLT approaches do indeed boost application performance significantly. On average, CoLT-SA achieves a 12% performance improvement, while CoLT-FA and CoLT-All achieve 14% improvements. On benchmarks like `Xalancbmk`, the performance improvements hover around 60% of runtime. Across other workloads like `Mcf`, `CactusADM`, `Astar`, `Omnetpp`, and `Bzip2`, at least one of the CoLT configurations improves performance over 10%. We anticipate that as applications with even larger working sets or virtualization are considered, these performance improvements will be even higher.

Figure 21 indicates that CoLT-SA, which has the simplest implementation, performs comparably to CoLT-FA and CoLT-All. Nevertheless, benchmarks like `Omnetpp` and `Bzip2` do see boosts from CoLT-FA/CoLT-All. Because we assume smaller 8-entry fully-associative TLBs, we expect CoLT-FA and CoLT-All results to be even higher with more realistically-sized superpage TLBs.

8. Conclusion

This paper proposes and designs Coalesced Large-Reach TLBs capable of exploiting address translation contiguity to achieve high reach. Due to a variety of OS memory management techniques involving buddy allocators, memory compaction, and superpaging, large amounts of translation contiguity are generated, even under heavy system load. While this contiguity typically cannot be exploited to generate superpages, CoLT provides lightweight hardware support to detect this behavior. As a result, large TLB miss eliminations are possible (on average, 40% to 58%), translating to performance improvements of 14% on average.

This work has a number of interesting implications for architects, system designers, and OS designers. We showcase TLB optimization techniques that architects can readily incorporate in existing processors. System designers and OS designers can tune their software systems to generate contiguity suitable for CoLT. For example, while superpages can overwhelm systems performance due to their overheads, CoLT provides alternate mechanisms to boost performance. We believe that CoLT will become even more critical as applications have increasingly large memory requirements and trends like virtualization become prevalent.

9. Acknowledgments

We thank the anonymous reviewers for their feedback. We also thank William Katsak for his help in modifying the Linux kernel for our studies. Finally, we thank Viji Srinivasan for her suggestions on improving the final version of the paper. The authors acknowledge the support of Rutgers University's Office of the Vice President for Research and Economic Development. This work was supported in part by their Faculty Research Grant award.

References

[1] "The Standard Performance Evaluation Corporation. SPEC CPU2006 Results," *http://www.spec.org/cpu2006*.
[2] K. Albayraktaroglu *et al.*, "BioBench: A Benchmark Suite of Bioinformatics Applications," *ISPASS*, 2005.
[3] AMD Corporation, "AMD Programmer's Manual," vol. 2, 2007.
[4] Andrea Arcangeli, "Transparent Hugepage Support," *KVM Forum*, 2010.
[5] T. Barr, A. Cox, and S. Rixner, "Translation Caching: Skip, Don't Walk (the Page Table)," *ISCA*, 2010.
[6] T. Barr, A. Cox, and S. Rixner, "SpecTLB: A Mechanism for Speculative Address Translation," *ISCA*, 2011.
[7] A. Basu, M. Hill, and M. Swift, "Reducing Memory Reference Energy with Opportunistic Virtual Caching," *ISCA*, 2012.
[8] R. Bhargava *et al.*, "Accelerating Two-Dimensional Page Walks for Virtualized Systems," *ASPLOS*, 2008.
[9] A. Bhattacharjee, D. Lustig, and M. Martonosi, "Shared Last-Level TLBs for Chip Multiprocessors," *HPCA*, 2010.
[10] A. Bhattacharjee and M. Martonosi, "Characterizing the TLB Behavior of Emerging Parallel Workloads on Chip Multiprocessors," *PACT*, 2009.
[11] A. Bhattacharjee and M. Martonosi, "Inter-Core Cooperative TLB Prefetchers for Chip Multiprocessors," *ASPLOS*, 2010.
[12] D. Bovet and M. Cesati, "Understanding the Linux Kernel," 2005.
[13] J. B. Chen, A. Borg, and N. Jouppi, "A Simulation Based Study of TLB Performance," *ISCA*, 1992.
[14] D. Clark and J. Emer, "Performance of the VAX-11/780 Translation Buffers: Simulation and Measurement," *ACM Transactions on Computer Systems*, vol. 3, no. 1, 1985.
[15] Z. Fang *et al.*, "Online Superpage Promotion with Hardware Support," *HPCA*, 2001.
[16] N. Ganapathy and C. Schimmel, "General-Purpose Operating System Support for Multiple Page Sizes," *USENIX*, 1998.
[17] Intel Corporation, "TLBs, Paging-Structure Caches and their Invalidation," *Intel Technical Report*, 2008.
[18] A. Jaleel *et al.*, "CMP$im: A Pin-based On-the-fly Multi-core Simulator," *4th Workshop on Modeling, Benchmarking, and Simulation*, 2008.
[19] G. Kandiraju and A. Sivasubramaniam, "Going the Distance for TLB Prefetching: An Application-Driven Study," *ISCA*, 2002.
[20] C.-K. Luk *et al.*, "Pin: Building Customized Program Analysis Tools with Dynamic Instrumentation," *PLDI*, 2005.
[21] C. McCurdy, A. Cox, and J. Vetter, "Investigating the TLB Behavior of High-End Scientific Appplications on Commodity Multiprocessors," *ISPASS*, 2008.
[22] D. Nagle *et al.*, "Design Tradeoffs for Software-Managed TLBs," *ISCA*, 1993.